MODULE 17 · DOMAIN 6 - TERRAFORM STATE MANAGEMENT

Drift, Refresh-Only, and the Refactoring Blocks

When reality moves, when you move things, and the blocks that put state changes into code.

DOMAIN 6 of 8 COURSE WEIGHT 12% MODULE 17 of 27

admin@sthithapragna.com

The refactoring blocks move state manipulation into your configuration, where a plan describes it and review catches it.

What drift is

- **Drift is the difference between state and reality.** Somebody changed something outside Terraform.
- **It is not the difference between configuration and state.** That is an ordinary plan.
- Terraform notices drift during the **refresh** that precedes a normal plan.
- It reports it separately, under **changes made outside of Terraform**, as module six described.
- Causes are usually mundane: an incident fix, a console click, or another tool with write access.

> ON THE EXAM
>
> Module four said a plan has three inputs. Drift is the gap between two of them, and the plan is the gap between the other two.

Two comparisons, two meanings

CONFIGURATION AGAINST STATE

The plan

- What you asked for, versus what exists
- Resolved by applying
- Shown as creates, updates, destroys
- This is the normal workflow

STATE AGAINST REALITY

The drift

- What Terraform believed, versus what is true
- Resolved by refreshing, or by reverting
- Shown under 'changes outside of Terraform'
- This is what refresh-only mode isolates

THE TIEBREAKER

A plan showing no changes with a long drift section is not a quiet day. It means reality moved and nothing has decided what to do.

Refresh-only mode

- **terraform plan -refresh-only** compares state against reality and proposes **no infrastructure changes**.
- **terraform apply -refresh-only** writes those findings into state, after you approve them.
- It is **unaffected by code changes**, so it purely detects manual modification.
- **It does not fix anything in your infrastructure.** It updates Terraform's belief to match what is true.
- It replaced the deprecated **terraform refresh** command, which did this with no preview and no prompt.

> ON THE EXAM
>
> Refresh-only accepts reality. A normal apply overrules it. Those are opposite responses to the same finding.

Three responses to drift

- **Revert it.** Run a normal apply. Terraform changes reality back to match your configuration.
- **Adopt it.** Run apply with -refresh-only. State accepts what happened, and your configuration is now out of date.
- **Codify it.** Edit the configuration to describe the new reality, then apply normally. The plan should be empty.
- **Adopting alone leaves configuration and reality disagreeing**, so the next normal apply reverts it anyway.
- So adopt and codify usually go together. Accept the change, then write it down.

> ON THE EXAM
>
> Only a human can decide which of these is right. Everything else about drift handling can be automated.

Detecting drift on a schedule

- A scheduled **plan -refresh-only** is a clean drift detector. It cannot propose infrastructure changes.
- Pair it with **-detailed-exitcode**: zero means no drift, two means drift was found.
- That gives a pipeline something to branch on, from module five's automation flags.
- The only step needing a human is **deciding which of the three responses applies**.
- HCP Terraform automates the detection side as **drift detection** and **health assessments**. Module twenty-three.

> ON THE EXAM
>
> Refresh-only plus detailed exit code is the whole detector. It is safe to schedule because it cannot change infrastructure.

The refactoring blocks

- **moved**, from Terraform 1.1. An object kept its identity but changed address.
- **import**, from Terraform 1.5. An existing object should come under management. Module nineteen.
- **removed**, from Terraform 1.7. An object should leave state, with or without being destroyed.
- All three are **declarative equivalents of state CLI commands**, and all three appear in the plan.
- That means they go through **code review**, which is the whole reason they exist.

> ON THE EXAM
>
> Three blocks, three verbs: rename, adopt, forget. Each has a CLI counterpart that does the same thing without a preview.

The three blocks, and their CLI equivalents

BLOCK	THE COMMAND IT REPLACES
moved (1.1)	terraform state mv
import (1.5)	terraform import
removed, destroy = false (1.7)	terraform state rm
removed, destroy = true (1.7)	Deleting the resource block and applying
No block for it	terraform state list, show, pull, push

ON THE EXAM

The last row matters: inspection has no declarative equivalent, which is why the state commands are still needed. Module eighteen.

The moved block

- **moved** with a **from** and a **to** address. Terraform updates state instead of destroying and recreating.
- Covers renaming a resource, moving one into a module, and **changing between count and for_each**.
- The plan shows a **move** rather than a create and a destroy, and no infrastructure is touched.
- **Without it, a rename is a destroy and a create.** Terraform compares addresses, not intentions.
- Module ten's migration problem, solved: nine databases renamed rather than rebuilt.

> ON THE EXAM
>
> This is the answer whenever a purely cosmetic refactor produces a destructive plan. The addresses changed, so tell Terraform they moved.

How long to keep a moved block

- **Delete it too early and you strand anyone who has not applied yet.** Their state still has the old address.
- Keep it until **every environment and every workspace** has applied at least once.
- HashiCorp's position is that it is **fine to keep moved blocks indefinitely**.
- **Especially in a shared module**, where you cannot know when every consumer has caught up.
- A moved block whose from address is already gone is simply a no-op.

> ON THE EXAM
>
> In a published module, keeping the moved block is the safe default. You have no visibility of who is still on the old version.

The removed block

- **removed** with a **from** address. It instructs Terraform to remove that resource from state.
- **You must delete the resource block first.** A removed block replaces it, rather than sitting alongside it.
- It works on a **module address** too, removing everything that module created.
- The plan says the object **will no longer be managed by Terraform**, and the summary counts stay at zero.
- Applying it changes **state**, even though it reports no infrastructure actions.

> ON THE EXAM
>
> A plan reporting zero to add, zero to change and zero to destroy is still a plan that changes state. Read the sentences, not just the summary.

The argument that decides everything

- **A removed block requires a nested lifecycle block.** Omitting it is a validation error.
- **destroy = true removes it from state and destroys the real infrastructure.**
- **destroy = false removes it from state and leaves the object alone.**
- That is the difference between a safe refactor and a real deletion, in one boolean.
- The documented use for false: **handing management to another tool or team**.

> ON THE EXAM
>
> Read the destroy argument before approving any plan containing a removed block. Nothing else in the block changes the outcome.

moved or removed?

MOVED

Same object, new address

- Terraform keeps managing it
- State is updated, nothing is touched
- For renames and restructuring
- Keep the block for a long time

REMOVED

Terraform stops managing it

- The resource block must be deleted
- State loses the binding
- For handing over, or splitting apart
- Delete the block once it has propagated

THE TIEBREAKER

moved keeps a binding and changes its name. removed deletes the binding. Only one of them can also delete infrastructure.

A word on prevent_destroy

- Module eleven established that **prevent_destroy dies with its resource block**.
- **A removed block with destroy = false is the correct way** to stop managing something without destroying it.
- It is declarative, it appears in the plan, and it survives review.
- Simply deleting a protected resource block, by contrast, produces a destroy plan.
- So the answer to 'keep the object, stop managing it' is a **removed block**, not a lifecycle rule.

> ON THE EXAM
>
> Module eleven posed this problem and deferred the answer. This is it, and it did not exist before Terraform 1.7.

Blocks or state commands?

THE BLOCKS

In configuration

- Appear in the plan before applying
- Reviewed like any other change
- Repeatable across environments
- Available from 1.1, 1.5 and 1.7

THE COMMANDS

At the terminal

- Mutate state immediately
- No plan, no preview, no review
- Run once, on one person's machine
- Still necessary in some situations

THE TIEBREAKER

Prefer the blocks. The commands remain the answer when you need to inspect state, or fix something the blocks cannot express.

Reading the scenario

THE WORDING SAYS	IT IS POINTING AT
Somebody changed it in the console	Drift. Then decide: revert, adopt or codify
Record what happened, change nothing	apply -refresh-only
Detect manual changes nightly	plan -refresh-only with -detailed-exitcode
A rename produced a destroy plan	A moved block
Another team should own this now	A removed block with destroy = false
Zero to add, change and destroy, but state changed	A removed block. The summary counts infrastructure

ON THE EXAM

Ask whether reality moved or you moved something. Drift is the first case; the refactoring blocks are the second.

> During an incident, an engineer increased a database's allocated storage through the cloud console. The change was correct and the incident is resolved. The Terraform configuration still specifies the old size.
>
> The team wants to keep the larger storage and have Terraform manage it correctly going forward.

What should they do?

A. Run terraform apply, which will reconcile state with reality

B. Run apply -refresh-only, then update the configuration to the new size

C. Run apply -refresh-only only. State now matches reality, so the work is done

D. Use a removed block so Terraform stops managing the database

◆ PAUSE AND DECIDE

What should they do?

A Just apply

Reverts the change. A normal apply makes reality match configuration, and the configuration still says the old size.

B Refresh-only, then codify

Correct. Adopt the change into state, then write it into configuration so the next normal apply proposes nothing.

C Refresh-only alone

State matches reality and configuration does not. The next normal apply would shrink the database back.

D A removed block

Stops Terraform managing the database entirely, which throws away management to avoid editing one attribute.

THE SIGNAL

Adopt and codify are two steps. Doing only the first leaves a change that the next routine apply will quietly undo.

A team is handing a legacy storage bucket over to a different department, who will manage it with their own tooling. The bucket must keep existing with all its contents.

An engineer deletes the resource block and writes a removed block with the from address, then runs plan. Terraform reports a validation error.

What is missing, and what must it say?

A. A to address, since removed blocks work like moved blocks

B. A nested lifecycle block with destroy = false

C. A prevent_destroy lifecycle rule on the original resource

D. The resource block must stay, with the removed block alongside it

◆ PAUSE AND DECIDE

What is missing, and what must it say?

A. A to address

removed has no destination. The object is leaving management, not moving. Only moved takes a to address.

B. lifecycle with destroy = false

Correct. The nested lifecycle block is required, and destroy = false is what forgets the object without deleting it.

C. prevent_destroy

Module eleven: it dies with the resource block, which has already been deleted. It also cannot survive to protect anything here.

D. Keep the resource block

Backwards. A removed block replaces the resource block; leaving both would be contradictory.

THE SIGNAL

Terraform requiring the lifecycle block is a safety feature. It refuses to let you leave the destroy decision implicit.

> Following module ten's advice, a team converts nine production databases from count over a list to for_each over a set. The names are unchanged. The plan proposes to destroy all nine and create nine replacements.
>
> They need the refactor to touch no infrastructure at all.

What do they add?

A. A removed block per database, with destroy = false, then re-import them

B. A moved block per instance, mapping each old index to its new key

C. prevent_destroy on each database, so the destroy is blocked

D. An ignore_changes rule so the address change is not detected

◆ PAUSE AND DECIDE

What do they add?

A Remove then re-import

Reaches the same state by a longer, riskier route, with a window where nine production databases are unmanaged.

B moved blocks

Correct, and exactly what moved exists for. Each block maps one old index to its new key, and no infrastructure is touched.

C prevent_destroy

Blocks the plan rather than fixing it. Terraform would refuse to apply, and the refactor would stay stuck.

D ignore_changes

Ignores attribute changes, not addresses. Instance identity is not an attribute, so there is nothing for it to ignore.

THE SIGNAL

moved blocks are per instance here, not per resource, because the change is to how instances are keyed.

> A pipeline runs terraform plan -refresh-only nightly against twelve configurations, with -detailed-exitcode, and raises a ticket when the exit code is 2. It has run for six months.
>
> A new engineer asks whether this could accidentally change production, since it runs unattended every night.

What is the correct answer?

A. Yes. Refresh-only writes findings into state, which is a change

B. No. It is a plan, so it changes nothing, and refresh-only cannot propose infrastructure changes anyway

C. Yes, if drift is found, since Terraform reverts it automatically

D. No, but only because -detailed-exitcode suppresses the apply step

◆ PAUSE AND DECIDE

What is the correct answer?

A — It writes to state

Confuses plan with apply. A refresh-only APPLY writes findings to state; a refresh-only PLAN only reports them.

B — No, on both counts

Correct, and doubly safe. Plan never changes anything, and refresh-only mode cannot propose infrastructure changes even when applied.

C — It reverts automatically

Terraform never applies anything unprompted. Reverting drift requires somebody to run a normal apply and approve it.

D — The exit code flag

That flag only changes the process exit code. It has no effect on what the command does.

THE SIGNAL

Two independent reasons this is safe. It is a plan, and it is refresh-only. Either alone would be sufficient.

> A shared module renames an internal resource and ships a moved block. Six months later, believing everyone has upgraded, the maintainer deletes the moved block and releases a new version.
>
> A team that had been pinned to a much older version upgrades directly to the newest, and their plan proposes to destroy and recreate the resource.

What went wrong?

A The consumer skipped versions, so they must upgrade one release at a time

B The moved block was deleted before every consumer had applied it. It should have been kept

C moved blocks only work within the same major version of a module

D The consumer's state is corrupt and must be rebuilt with import

◆ PAUSE AND DECIDE

What went wrong?

A Skipped versions

There is no requirement to upgrade incrementally, and a well-maintained module should not need it. It also only works if the block still exists.

B Deleted too early

Correct. The consumer's state still holds the old address, and nothing remains to explain the rename. HashiCorp says keeping them indefinitely is fine.

C Major version limit

Invents a rule. A moved block is evaluated against whatever state exists, with no relationship to version numbering.

D Corrupt state

The state is perfectly valid. It simply records the old address, which is exactly what the moved block existed to translate.

THE SIGNAL

In a published module you cannot know who is still behind. That uncertainty is the argument for keeping moved blocks permanently.

Module 17 recap

- **Drift is state against reality.** The plan is configuration against state. Different comparisons.
- **Three responses: revert, adopt, codify.** Adopting alone leaves the next apply to undo it.
- **plan -refresh-only with -detailed-exitcode** is a safe scheduled drift detector.
- **moved renames, removed forgets, import adopts.** All three appear in the plan.
- **A removed block requires a lifecycle block**, and destroy = false is what keeps the object.
- **Keep moved blocks for a long time**, and indefinitely in a shared module.

> ON THE EXAM
>
> The blocks exist so state changes go through a plan and a review, instead of one person's terminal.